CLUBROOF

Technische Erstprüfung

Befunde, Nachweise und nächste Entwicklungsschritte

Stand der Prüfung: 19. Oktober 2026
Grundlage: hochgeladenes GitHub-Quellcodearchiv der ClubRoof-App

Gesamturteil

Der Stand enthält eine umfangreiche, strukturiert aufgebaute Vereins-App mit gemeinsamem TypeScript-Code, eigenem Backend, Datenbankmigrationen und zahlreichen automatisierten Prüfungen. Der Web-Export lässt sich bauen. Vor einem Pilotbetrieb mit echten Mitgliederdaten und realer Kassenführung sollten die vier als P1 eingestuften Fehler behoben werden. Die Ergebnisse rechtfertigen keinen vollständigen Neubau; sie sprechen für gezielte Stabilisierung der vorhandenen Abläufe.

Prioritäten auf einen Blick

ID	Prio.	Befund	Nachweis
F01	P1	Beitrittsanfragen können mit Personen außerhalb des erlaubten Bereichs verknüpft werden.	Quellcode
F02	P1	Nach automatischer Abmeldung bleiben Daten des vorherigen Kontos im Cache.	Modultest
F03	P1	Stornierte Beitragszahlungen gelten weiterhin als bezahlt.	API + DB
F04	P1	Parallele Zahlungen können dieselbe Forderung überbuchen.	API + DB
F05	P2	Abwesenheiten und Terminverschiebungen erzeugen falsche Teilnahmestatus.	API + DB
F06	P2	Temporäre Serverfehler löschen vorgemerkte Rückmeldungen.	Modultest
F07	P2	Die Pflicht zur 2FA-Einrichtung wird nur auf ausgewählten URL-Pfaden durchgesetzt.	Quellcode

P1: vor echten Mitgliederdaten bzw. realer Kassenführung beheben. **P2:** wichtig für zuverlässigen und konsistent geschützten Betrieb. Diese Prioritäten sind eine technische Einordnung, keine formale Sicherheitszertifizierung.

Was dieser Bericht abdeckt

Geprüft wurden Architektur und Projektstand, zentrale Rechte- und Sitzungspfade, Beiträge/Kassenbuch, Abwesenheiten und Offline-Rückmeldungen. Die vorhandenen Tests wurden ausgeführt; gezielte Zusatzprüfungen stellen die wichtigsten funktionalen Fehler nach. Native Gerätebedienung, ein vollständiger visueller Rundgang und ein produktives Hosting waren nicht Gegenstand dieser Erstprüfung.

1. Identitäten und Verwaltungsrechte

F01 | P1 | Hohe Sicherheit der statischen Feststellung

Bestehende Personen werden nicht ausreichend autorisiert verknüpft

Beim Freigeben einer Beitrittsanfrage prüft der Server, ob die handelnde Person die Mannschaft der Anfrage verwalten darf. Die zusätzlich übergebenen bestehenden Personen werden anschließend jedoch nur über ihre ID und Vereinszugehörigkeit aufgelöst. Eine entsprechende Prüfung der Mannschaftsgrenzen oder der Rollen der zu verknüpfenden Person fehlt.

Damit kann eine berechtigte Person mit eingeschränktem Mannschaftsumfang eine Anfrage mit einer Person aus einem anderen Bereich desselben Vereins verbinden. Hat die bestehende Person noch keinen App-Zugang, wird ihr das neue Benutzerkonto zugeordnet; ihre vorhandenen Rollen bleiben erhalten. Falls sie bereits weitergehende Verwaltungsrechte hat, ist somit auch eine Rechteausweitung möglich. Die Verknüpfung eines bestehenden Kindes wird ebenfalls nicht ausreichend gegen den erlaubten Bereich abgesichert.

Bedingungen und Grenzen: Der Pfad erfordert bereits ein Konto mit Einladerechten, eine passende Beitrittsanfrage und die Kenntnis der Ziel-Personen-ID. Es ist kein nachgewiesener Zugriff zwischen zwei Vereinen und kein passwortloser Zugriff durch beliebige Außenstehende. Der relevante Fehler liegt innerhalb eines Vereins.

Korrektur und Abnahme

Das Einrichten eines Zugangs und das Verknüpfen bestehender Personen benötigen eine zentrale Objekt- und Rollenprüfung. Ein Einladerecht für eine Mannschaft darf keine Identität mit weitergehenden Rechten verfügbar machen. Für die Eltern-Kind-Zuordnung sind ebenfalls ein zulässiger fachlicher Bezug und geeignete Bestätigung erforderlich. Regressionen sollten fremde Mannschaft, Person mit höheren Rechten, bestehendes Kinderkonto und reguläre zulässige Verknüpfung abdecken.

`apps/api/src/services/invitations.ts:566-586, 597-604, 655-707`

`apps/api/src/actor.ts:60-83, 107-117`

`apps/api/src/services/profiles.ts:157-219, 230-243`

F07 | P2: 2FA-Pflicht hängt am URL-Pfad

Die zentrale Authentifizierung sperrt bei fehlender verpflichtender 2FA-Einrichtung nur bestimmte Pfadpräfixe, etwa /admin und /invitations. Andere privilegierte Routen, darunter die Verwaltung von Mannschafts-Einladungslinks unter /teams, fallen nicht darunter. Ihre Dienste prüfen das Einladerecht, aber nicht dieselbe 2FA-Vorgabe. Das gilt für die Durchsetzung der Einrichtungspflicht; es belegt keinen Umgehungsweg eines bereits korrekt eingerichteten zweiten Faktors.

Korrektur: Die Vorgabe an die geschützte Aktion oder ein einheitliches Berechtigungsmerkmal der Route binden. Eine Pfadliste ist bei neuen Verwaltungsfunktionen leicht unvollständig. Abnahme: alle betroffenen Schreibaktionen bei fehlender Einrichtung ablehnen, Einrichtung selbst weiterhin ermöglichen.

`apps/api/src/app.ts:101-120`

`apps/api/src/routes/invitations.ts:50-64`

`apps/api/src/services/invitations.ts:333-372`

2. Sitzungen und Offline-Verhalten

F02 | P1 | Mit unverändertem Originalcode isoliert reproduziert

Ein anderes Konto übernimmt vertrauliche Cache-Daten

Der ausdrücklich ausgelöste Logout leert Abfrage-Cache, Offline-Speicher und Outbox. Bei einer automatischen Abmeldung durch HTTP 401 werden dagegen nur Token und Sitzungsstatus zurückgesetzt. Auch die anschließende Anmeldung bzw. Sitzungsübernahme bereinigt den alten Cache nicht. Abfrageschlüssel wie [event, id] enthalten keine Benutzerkennung.

Im Modultest lädt Trainer A Termindaten mit einem vertraulichen Absagegrund. Nach HTTP 401 meldet sich Spieler B an derselben App-Instanz an. Der echte QueryClient liefert B weiterhin den Trainer-Datensatz; innerhalb der 30 Sekunden Frischezeit erfolgt dabei keine neue API-Anfrage. Ein weiterer Test zeigt, dass dieser Datensatz sogar unter B im Offline-Speicher abgelegt und später wiederhergestellt werden kann.

Auswirkung: Ein lokales Datenleck beim Benutzerwechsel auf demselben Gerät. Die serverseitige Filterung der Absagegründe kann diesen bereits ausgelieferten Cache-Datensatz nicht nachträglich korrigieren. Das Problem besteht schon im Einvereinsbetrieb.

Zusätzliche Folge: Beim Anmelden starten das asynchrone Laden der kontoeigenen Outbox und ein sofortiger Sendeversuch getrennt. Noch im Speicher liegende Antworten von A können deshalb mit dem API-Kontext von B gesendet werden. Ob der Server sie akzeptiert, hängt von B ab; der falsche Sendekontext ist im Modultest belegt.

Korrektur und Abnahme

Eine gemeinsame Bereinigung für Logout, HTTP 401 und Kontowechsel einführen; alte Abfragen und Synchronisierungen abbrechen. Cache und persistierte Daten nach Benutzer und künftig Verein trennen. Outbox erst nach abgeschlossenem kontobezogenem Laden senden und den Eigentümer vor jedem Versand prüfen. Regression: Trainerdaten laden, Sitzung verlieren, als Spieler anmelden und sowohl Online- als auch Offline-Ansicht prüfen.

`apps/mobile/src/lib/session.tsx:79-95, 113-142`

`apps/mobile/src/lib/offline-cache.ts:48-67`

`apps/mobile/src/app/events/[id].tsx:98-108, 450-452`

`apps/mobile/src/components/outbox-sync.tsx:42-52`

F06 | P2: Serverfehler entfernen wartende Rückmeldungen

Die Outbox behält eine Antwort nur bei einem Netzwerkfehler mit Status 0. Auch HTTP 429, 500 oder 503 werden als endgültige Ablehnung behandelt und der Eintrag entfernt. Im Test mit HTTP 503 bleibt die Warteschlange leer, obwohl eine spätere Übermittlung möglich wäre. Ein Toast meldet den Fehler, die Antwort muss jedoch neu eingegeben werden.

Korrektur: Vorübergehende Fehler erhalten und mit begrenzten Wiederholungen behandeln; bei HTTP 401 zunächst neu anmelden lassen. Nur endgültige fachliche Ablehnungen, etwa eine abgelaufene Frist, aus der Warteschlange entfernen.

`apps/mobile/src/lib/outbox.ts:87-106`

3. Beiträge und Kassenbuch

F03 und F04 | P1 | Gegen echte lokale PostgreSQL-Datenbank reproduziert

F03: Zahlung storniert, Forderung bleibt bezahlt

Die Beitragsübersicht summiert alle zu einer Forderung gespeicherten Zahlungen. Sie prüft nicht, ob deren Kassenbuchungen inzwischen storniert sind. Der Stornodienst setzt den Status der Buchung, verändert aber die fachliche Zahlungssumme der Forderung nicht.

Prüfschritt	Tatsächlich beobachtet
50 EUR Forderung vollständig bezahlen	Bezahlt: 50 EUR; offen: 0 EUR
Zugehörige Kassenbuchung stornieren	Buchung ist nachweislich storniert.
Beitrag erneut laden	Weiterhin bezahlt: 50 EUR; offen: 0 EUR
Korrigierte Zahlung erfassen	HTTP 400: overpayment
Forderung selbst stornieren	HTTP 409: has_payments

Auswirkung: Kassenbuch und offene Beiträge widersprechen sich. Eine korrigierte Ersatzbuchung und auch die vorgesehene Stornierung der Forderung werden blockiert. Ebenso müssen Jahressummen der Beiträge den Stornierungsstatus berücksichtigen.

Korrektur: Fachliche Zahlungssummen aus aktiven zugehörigen Kassenbuchungen ableiten oder ein durchgängiges Zahlungsstornomodell einführen. Die Historie erhalten. Abnahme: zahlen, Buchung stornieren, Forderung wieder offen sehen, anschließend erneut korrekt zahlen.

`apps/api/src/services/club-fees.ts:75-91, 181-190, 639-651, 737-745`

`apps/api/src/services/club-cash.ts:519-570`

F04: Parallele Zahlungen überbuchen eine Forderung

Der offene Betrag wird vor der Schreibtransaktion gelesen und geprüft. Zwei gleichzeitig eintreffende Anfragen können dadurch beide denselben noch offenen Restbetrag verwenden. Die Transaktion um die anschließenden Inserts verhindert diesen Ablauf nicht.

Beobachtung: Bei einer Forderung über 50 EUR wurden zwei Zahlungen über jeweils 50 EUR mit HTTP 201 angenommen. Danach bestanden zwei Zahlungen und aktive Kassen-Einnahmen über insgesamt 100 EUR. Die Forderung selbst betrug weiterhin 50 EUR.

Nachweismethode: Zwei unveränderte Datenbank-SELECTs wurden nach dem Lesen zurückgehalten, bis beide Ergebnisse vorlagen. Danach liefen beide Original-Requests weiter. Das stellt eine zulässige gleichzeitige Ausführungsreihenfolge kontrolliert her; SQL, gelesene Werte und Produktionscode wurden nicht geändert.

Korrektur: Die Forderung innerhalb derselben Transaktion sperren, dann den aktuellen Restbetrag prüfen und buchen. Wiederholte identische Client-Anfragen zusätzlich über einen Idempotenzschlüssel absichern. Abnahme: zwei parallele Vollzahlungen dürfen nur eine wirksame Zahlung ergeben.

`apps/api/src/services/club-fees.ts:639-651, 680-704`

`packages/db/src/schema/operations.ts:995-1012`

4. Abwesenheiten und Teilnahme

F05 | P2 | Zwei unterschiedliche Abläufe über die Original-API bestätigt

Fall A: Zwei Abwesenheiten, eine wird gelöscht

Automatische Absagen sind mit genau einer Abwesenheit verknüpft. Beim Anlegen einer weiteren überschneidenden Abwesenheit überspringt die Logik bereits abgesagte Personen. Wird die zuerst verknüpfte Abwesenheit gelöscht, setzt der Dienst die Teilnahme auf den Standardstatus zurück, ohne die zweite noch gültige Abwesenheit auszuwerten.

Zustand im Test	Teilnahme des Spielers
Ausgangszustand im Team mit aktiver Rückmeldung	Offen (pending)
Urlaub A und Krankheit B umfassen denselben Termin	Abgesagt (no), mit A verknüpft
Nur Urlaub A wird gelöscht; Krankheit B besteht weiter	Falsch: wieder offen, keine absenceId
Trainer lädt den Termin erneut	Falsch: absent=false

Bei Mannschaften mit automatischer Zusage setzt derselbe Code auf yes zurück. Dort kann eine weiterhin abwesende Person sogar wieder als zugesagt zählen. Dies ist aus dem Kontrollfluss ableitbar; der konkrete Datenbanktest verwendete das aktive Antwortmodell und beobachtete pending.

`apps/api/src/services/absences.ts:84-118, 120-142, 313-315`

Fall B: Ein Termin wird in eine bestehende Abwesenheit verschoben

Bei der Neuanlage eines Termins werden Abwesenheiten berücksichtigt. Beim Ändern des Termindatums wird der automatisch abgeleitete Teilnahmestatus dagegen nicht neu berechnet.

Beobachtung: Ein zunächst freies Training wurde über die Original-PATCH-Route auf den 10.11.2026 verschoben. Für diesen Tag bestand bereits die Krankheit B. Die Teilnahme blieb trotzdem pending; absenceId blieb leer. Der neue Veranstaltungstag und die gespeicherte Verfügbarkeit widersprechen sich damit.

`apps/api/src/services/event-admin.ts:245-302, 576-611`

Gemeinsame Korrektur

Einen zentralen Dienst zur Neuberechnung automatisch abgeleiteter Teilnahmen verwenden. Er muss alle gültigen Abwesenheiten einer Person anhand von Datum, Vereinszeitzone und Mannschaft auswerten. Diesen Dienst beim Anlegen, Ändern und Löschen von Abwesenheiten sowie beim Verschieben von Terminen aufrufen.

Manuell abgegebene Zu- und Absagen brauchen eine bewusst festgelegte Produktregel. Sie sollten von automatisch durch Abwesenheit erzeugten Statuswerten unterscheidbar bleiben, damit eine Neuberechnung keine absichtliche Benutzerentscheidung überschreibt.

Abnahmeszenarien

Überschneidung A/B mit Löschen oder Verkürzen von A; automatische Zusage und aktive Rückmeldung; teambezogene Abwesenheit; Termin in einen Zeitraum hinein und aus ihm heraus verschieben; manuelle Absage unverändert erhalten; Termine an lokalen Tagesgrenzen prüfen.

5. Durchführung und Testergebnisse

Nachweise und Grenzen der Aussage

Vorhandene Prüfungen

Bereich	Gesamt	Bestanden	Fehler	Überspr.
Fachliche Grundlagen	38	38	0	0
Design- und Farbvarianten	936	936	0	0
Datenbank-Demodaten	11	11	0	0
API und API-Hilfsdienste	261	261	0	0

Ergebnis des letzten Laufs: 1246 von 1246 Tests bestanden. Die große Zahl enthält viele parametrisierte Design-/Farbtests; sie bedeutet keine gleich große Anzahl unabhängiger fachlicher Abläufe.

Zusätzlich bestanden: Lint und TypeScript-Prüfung in allen fünf Codepaketen, projektspezifische Prüfungen für Übersetzungen, zugängliche Beschriftungen, Farbvorgaben und Desktop-Vorlagen. Der Expo-Web-Export wurde erfolgreich gebaut. Diese statischen Prüfungen ersetzen keinen vollständigen visuellen oder barrierefreien Bedienungstest.

Im ersten vollständigen API-Lauf bestanden 259 von 261 Tests; zwei Tests überschritten das Standardlimit von fünf Sekunden. Der dokumentierte letzte Lauf wiederholte die unveränderte Suite mit einem Test-Zeitlimit von 20 Sekunden. Das Zeitlimit wurde nur als CLI-Option gesetzt, nicht im Projekt geändert.

Gezielte Zusatzprüfungen

API + echte Datenbank: vier bestätigte Abläufe zu Zahlungsstorno, parallelen Zahlungen, überlappenden Abwesenheiten und Terminverschiebung. Verwendung ausschließlich eigener lokaler Testdatenbanken, mitgelieferter Demodaten und fester Uhr. API-Aufrufe erfolgten über Fastify inject; E-Mail, Push und Dateispeicher waren Testadapter.

Frontend-Modultests: Originalcode im Speicher transpiliert, echter installierter QueryClient und QueryObserver; React-Hooks sowie Plattform-/Speicherzugriffe isoliert simuliert. Bestätigt wurden Cache-Übernahme nach HTTP 401, Speicherung unter dem falschen Konto, falscher Outbox-Sendekontext und Entfernen einer Antwort bei HTTP 503. Kein nativer Gerätetest.

Nicht abschließend verifiziert

F01 und F07 wurden anhand der betroffenen Autorisierungs- und Datenpfade bewertet. Der ergänzende automatisierte Test zur Rechteausweitung bei F01 wurde durch eine automatische Sicherheitsprüfung blockiert und nicht durchgeführt. Die übrigen Prüfungen liefen weiter; für F01 wird ausdrücklich kein erfolgreicher Laufzeit-Angriffstest behauptet.

Nicht geprüft: produktives Hosting, Backup-Wiederherstellung, echte Push-Zustellung, iOS-/Android-Builds und Geräteverhalten, vollständige Browserbedienung, Last unter realem Betrieb oder rechtliche Konformität. Das ZIP liefert außerdem keine von mir verifizierte Git-Commit-ID.

Umgebung: Node.js 24.19.0, pnpm 11.25.0, PostgreSQL 16.15. Das Projekt nennt Node 22 und pnpm 10.28.0. Der abweichende lokale Prüflauf ersetzt daher nicht die bestehende CI mit den Projektversionen.

6. Entwicklungsstand und nächste Schritte

Vorhandene Grundlage gezielt stabilisieren

Was bereits eine gute Grundlage bildet

HTTP-Routen, Fachlogik, gemeinsame Verträge/Rechte, Datenbankschema und Gestaltung sind getrennt. Migrationen, Demodaten, Integrationstests und CI sind vorhanden. Die Struktur bietet eine brauchbare Grundlage; die Befunde betreffen konkrete Lücken in übergreifenden Abläufen.

`README.md; CLAUDE.md; apps/api/src/app.ts; apps/api/src/actor.ts`

`packages/core/src/; packages/db/src/; packages/db/drizzle/`

Ist-Architektur klar festhalten

Implementiert sind Fastify, PostgreSQL/Drizzle und Expo/React Native für iOS, Android und Web, einschließlich Desktop-Anpassungen. Eine separate Next.js-Verwaltung ist nicht enthalten. Ältere Planangaben zu Next.js/NestJS sollten an diesen Ist-Stand angepasst werden.

`apps/api/package.json; apps/mobile/package.json`

`apps/mobile/src/lib/layout.ts:7-15; apps/mobile/src/app/_layout.tsx`

`docs/PLAN.md:145-170`

Datenbank pro Verein ist derzeit nicht das umgesetzte Modell

Die App nutzt eine gemeinsame PostgreSQL-Datenbank mit club_id an fachlichen Tabellen. Eine eigene Datenbank pro Verein ist nicht umgesetzt. Die Vereinszuordnung muss bei diesem Modell in Abfragen und Verknüpfungen konsistent geschützt sein. Vor dem Mehrvereinsbetrieb das Modell bewusst festlegen und mit mindestens zwei getrennten Testvereinen prüfen.

`packages/db/src/client.ts:6-9; apps/api/src/actor.ts:53-58`

`packages/db/src/schema/; docs/FUNKTIONEN.md:265-274`

Ausdrücklich noch geplante Produkt- und Betriebsschritte

Mehrvereinsbetrieb, Pläne/Abo, Teile des Demo-Betriebs, Store-Reife und Testserver sind als offen dokumentiert. Push braucht noch die passende Expo-Projekt-/Build-Anbindung. Diese geplante Arbeit wurde nicht als Fehler einer fertigen Funktion bewertet.

`docs/FUNKTIONEN.md:184-194, 250-274; docs/OFFENE-PAKETE.md:20-31`

Empfohlene Reihenfolge

1. Konto-/Datentrennung und Kassenkonsistenz (F01-F04) beheben, Regressionen ergänzen.
2. F05-F07 korrigieren; Originalsuite und gezielte Grenzfälle gemeinsam prüfen.
3. Pilot mit Testdaten und klaren Rollen auf echten iOS-/Android-Geräten sowie am Desktop durchführen.
4. Mehrvereinsbetrieb fertigstellen, anschließend Abo-, Betriebs- und Store-Pakete umsetzen.

Alle Dateiverweise sind relativ zur Wurzel des hochgeladenen Repositorys. Abgleich mit dem Archiv: 861 Originaldateien geprüft, keine davon geändert. Dieser Bericht beschreibt den geprüften Upload und keine späteren GitHub-Änderungen.